Load Balancing Simulator Demo 2: Working Prototype

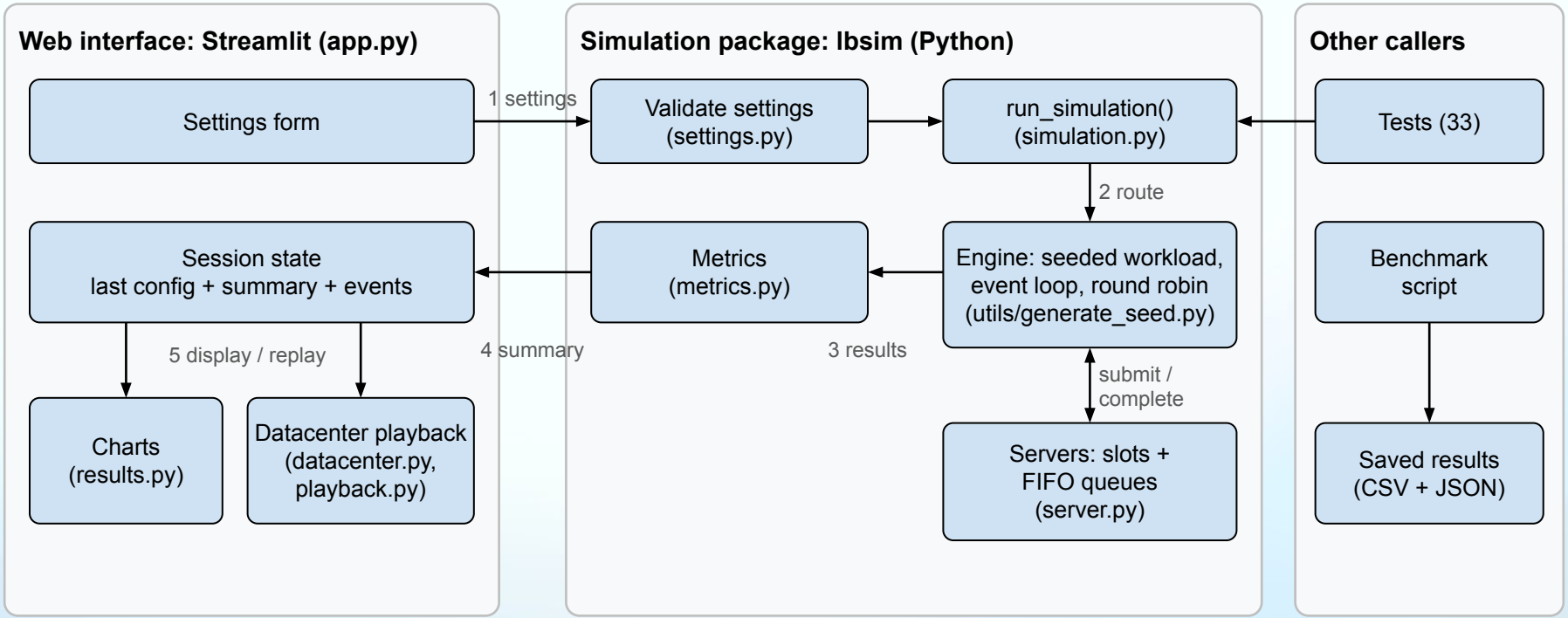
Daniel Thomas, Kimone Walker, Neev Jain, Sean O'Connor

Mentor: Shripad Nadgowda, Meta

Demo 2 Goals and Status

Commitment from our proposal	Status	Evidence
Working simulator, end to end	Done	Settings form → simulation → results
At least one routing policy	Done	Round robin (least-active-requests in Demo 3)
Basic interface and result viewing	Done	Charts, datacenter playback, server details
Correctness and replay checks	Done	33 automated tests; seeded runs repeat exactly
Baseline experiment outputs	Done	10,000 requests on 10, 100 and 1,000 servers
Code, design doc, slides and video on demo-2	Done	Final sync before October 21 at noon

Architecture and Implementation



Everything runs in one local Python process, no separate services yet
Server state is kept in memory, round robin doesn't read it, least-active-requests (Demo 3) will

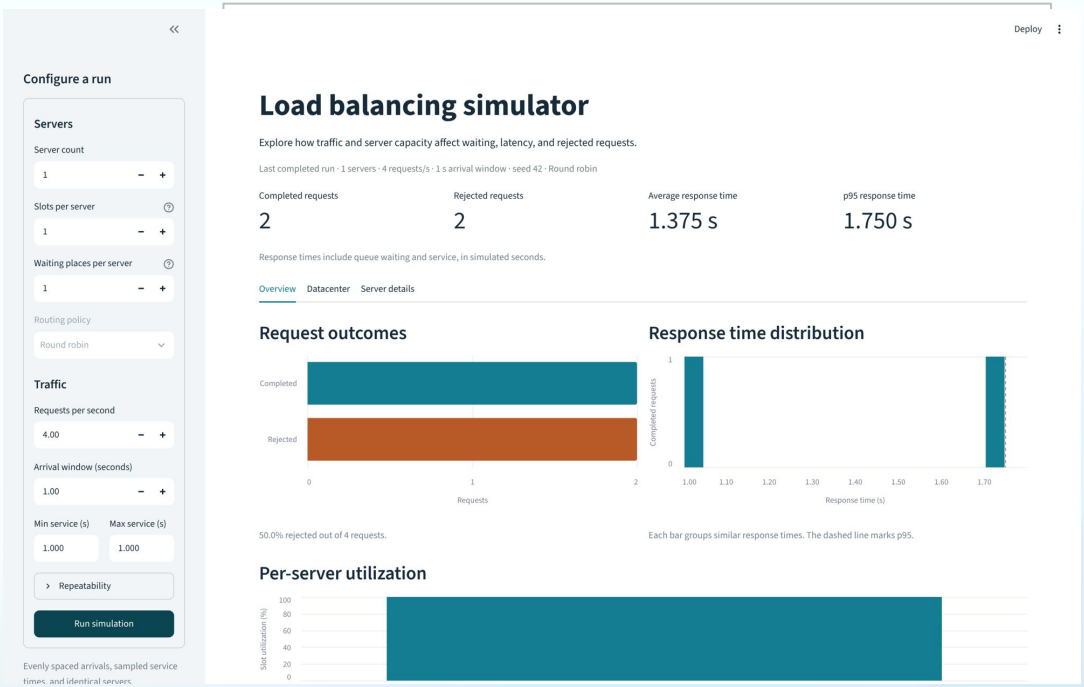
Live demo

Overload: one server receives four requests in one second

Playback: eight servers, scrub to the moment a queue fills

One Run, End to End

1. Configure servers and traffic
2. Validate every setting once
3. Generate the seeded workload
4. Route, run, queue or reject
5. Drain, then summarize
6. Show results and replay events



The running app: settings on the left, results on the right

Overload Example in the App

Load balancing simulator

Explore how traffic and server capacity affect waiting, latency, and rejected requests.

Last completed run · 1 servers · 4 requests/s · 1 s arrival window · seed 42 · Round robin

Completed requests	Rejected requests	Average response time	p95 response time
2	2	1.375 s	1.750 s

Response times include queue waiting and service, in simulated seconds.

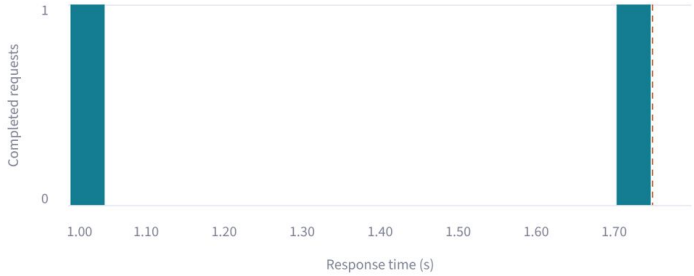
Overview | Datacenter | Server details

Request outcomes



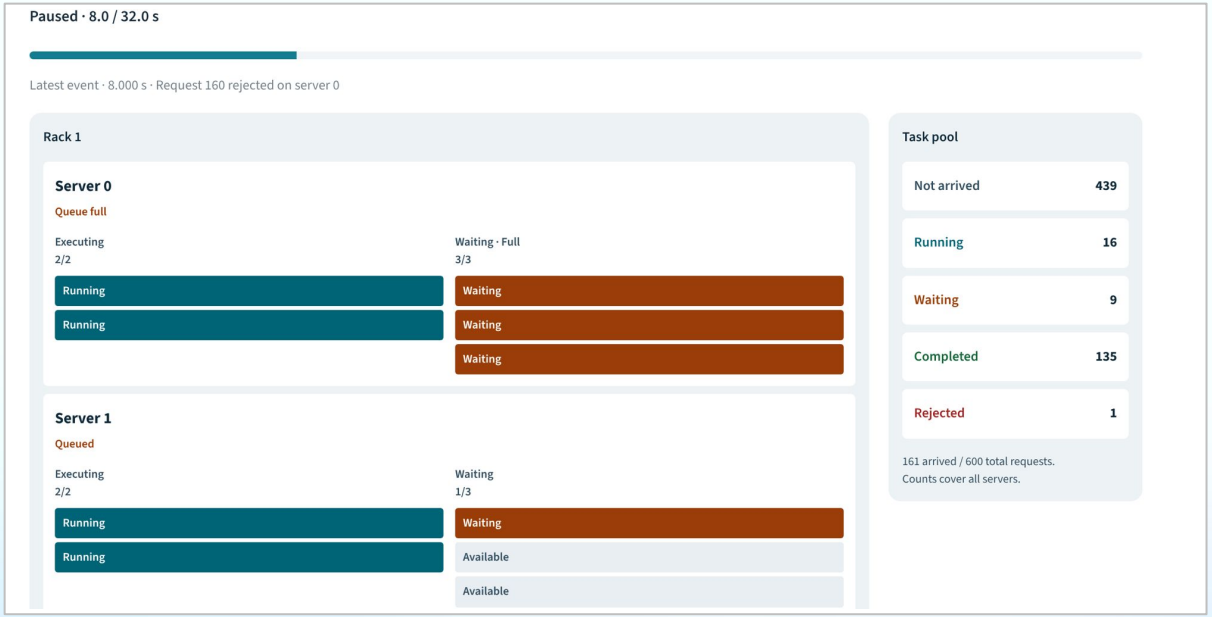
50.0% rejected out of 4 requests.

Response time distribution



Each bar groups similar response times. The dashed line marks p95.

Playback: Watching a Queue Fill



Metrics: 8 servers, 2 slots, 3 waiting places, 20 requests/s for 30s, seed 42

At t = 8.0 s, server 0 is full and request 160 is rejected

Servers 1-3 still have waiting places: round robin sends request i to server i mod 8 without checking load

Whole run: 586 completed, 14 rejected.

Results

Tech: 1 server, 1 slot, 1 waiting place, 4 requests/s for 1s, service 1s, seed 42

Time (s)	Event	Running / waiting
0	Request 0 starts (done at 1)	1 / 0
0.25	Request 1 queues	1 / 1
0.5	Request 2 rejected: queue full	1 / 1
0.75	Request 3 rejected: queue full	1 / 1
1	0 finishes; 1 starts at once	1 / 0
2	1 finishes; server idle	0 / 0

2
completed

2
rejected

1.375 s
mean response

1.75 s
p95 response

- Request 1 waits 0.75 s and runs 1s its response time is 1.75s
- Utilization is 100%
- With 0.1s service instead: 4 completed, 0 rejected, 0.100s mean
- 40% utilization

Baseline

One 10,000-request trace, three fleet sizes

- Metrics: 100 req/s for 100s, service 0.1-1.0 s, 1 slot, 10 waiting per server, seed 42, round robin

Servers	Median runtime	Peak memory	Completed	Rejected
10	0.075 s	35.4 MiB	1,905	8,095
100	0.135 s	37.8 MiB	10,000	0
1,000	0.128 s	38.7 MiB	10,000	0

What it shows

The whole run (generate, route, record, drain, summarize) stays under 0.14 s and 39 MiB here

Doesn’t Show

More servers also add capacity, this doesn’t isolate the cost of server count

How the Engine Keeps Time

Event clock

Time jumps to the next arrival or completion. No sleeps, no threads: 10,000 requests run in 0.08 - 0.14 s (median)

Completions first

At equal timestamps, completions run before arrivals, so a request arriving at t = 3 can use a slot freed at t = 3

Deterministic ties

Pending completions sort by (time, server, request). The same settings and seed produce the same events every time

Drain, then measure

Accepted work finishes after arrivals stop. Latency includes draining, utilization counts only the arrival window

33 Tests Check the Simulator

Every run, every event

Applied to normal, overload, multi-slot, zero-queue, mixed and empty runs

- Each request ends once: completed or rejected
- Never more than the slots or queue allow
- Each event is one valid state change
- FIFO start order on every server
- All servers idle after draining
- Same seed, identical output

Exact traces

Our documented one-server run and two-server runs, every event and metric checked by hand.

Unit tests

Eight more test files: server lifecycle, validation, routing, metrics, UI, playback.

Changes and Limitations

Updated Scope:

Changed from the plan

- Arrivals are evenly spaced, not bursty
- Server size and reliability fields do not affect runs yet

Planned for Demo 3

- Least-active-requests policy
- Saving, reloading and comparing runs

Known limits

- Utilization is slot occupancy, not CPU
- No size cap: huge runs can stall the UI
- Baseline: one machine, one seed

Next Steps

- **Demo 3 plan:**
  - Least-active-requests: send each request to the server with the fewest running requests, ties to the lowest ID
  - Compare both policies on the same workloads: low, near-capacity and overloaded traffic, plus 20% more traffic, three seeds each
  - Save, reload and replay runs
  - Profile the engine, fix one measured bottleneck, and rerun the benchmark
- **From our mentor meeting, to explore:**
  - Load feedback: each routing decision reads the servers' current load
  - What-if scenarios: a server fails, traffic spikes, the share of heavy requests grows

Questions?

Thank you!

---

github.com/ec528-fall26/load-balancing -> branch demo-2